

## scanf函数

格式化输入 从标准输入拷贝数据到自己的内存

### scanf

语法:

```
#include <stdio.h>
int scanf( const char *format, ... );
```

包含头文件

可变参数

字符串, 说明格式

返回一个整数

分区 新分区 1 的第 37 页

## scanf是不安全的

error C4996: 'scanf': This function or variable may be unsafe.

分区 新分区 1 的第 38 页

## scanf的流程

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main() {
    int i; //用来存放从stdin过来的数据
    int ret = scanf("%d", &i);
    // %d 是一个控制符, 用来说将要接收的数据的类型是整数
    // &i 将数据存入i的地址当中, &符号取地址, 不可省略
    printf("ret = %d, i = %d\n", ret, i);
}
```

Microsoft Visual Studio 调试控制台

1234  
ret = 1, i = 1234

abc  
ret = 0, i = -858993460

^Z  
^Z  
^Z  
ret = -1, i = -858993460

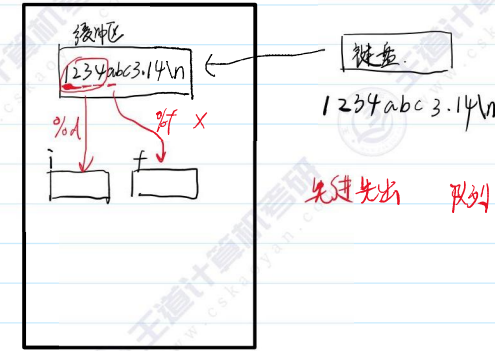
- 如果返回值是-1, 我们认为特殊情况, 叫 EOF

- 1、准备变量 —— 数据的目的地
- 2、scanf的两类参数
  - 第一类参数字符串, 用来描述数据的格式
  - 第二类用来说明存放数据的地址 —— 记得使用&运算符
- 3、scanf的返回值是读取成功的变量个数
  - 如果返回值和%符号个数相等 —— 读取成功
  - 如果返回值小于%符号个数 —— 有数据读取失败

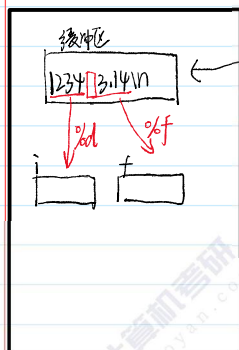
分区 新分区 1 的第 39 页

## stdin的性质

```
int i;
float f;
scanf("%d%f", &i, &f);
printf("i = %d, f = %f\n", i, f);
```



分区 新分区 1 的第 40 页



1234 3.14\n  
1234 \n3.14\n

scanf %d %f %lf 忽略缓冲区里面的前置空白字符 (换行、空格、制表)

我们使用scanf的时候, 我不需要考虑输入内容当中有多少个空格, 控制字符可以直接连着写

分区 新分区 1 的第 41 页

scanf %c

读取一个char

```
char ch;
scanf("%c", &ch);
printf("ch = %c\n", ch);
```

scanf %c不会忽略前面的空白字符

```
int i;
char ch;
scanf("%d%c", &i, &ch);
printf("i = %d, ch = %c\n", i, ch);
```

Microsoft Visual Studio 调试控制台  
1234 a  
i = 1234, ch =

分区 新分区 1 的第 42 页

混合输入

我们希望改变scanf %c的性质, 让它能够忽略前面的空白字符

```
int i;
char ch;
scanf("%d %c", &i, &ch); // %c前面的空格, 可以让%c忽略前面的空白
printf("i = %d, ch = %c\n", i, ch);
```

Microsoft Visual Studio 调试控制台

1234  
a  
i = 1234, ch = a

一般的使用场景

绝大多数情况下 scanf的格式字符串里面, 是只有%控制符, 偶尔会在%c前面加空格

分区 新分区 1 的第 43 页

getchar

getchar

语法:

```
#include <stdio.h>
int getchar( void );
```

getchar()函数从STDIN(标准输入)获取并返回下一个字符, 如果到达文件尾返回EOF.

等价于 scanf %c, 也不会忽略空白字符

```
char ch1, ch2;
ch1 = getchar();
ch2 = getchar();
printf("ch1 = %c, ch2 = %c\n", ch1, ch2);
```

选择 Microsoft Visual Studio 调试控制台

a b  
ch1 = a, ch2 =

分区 新分区 1 的第 44 页

对应 p11 内容

### ①程序代码：

```
#include <stdio.h>

int main() {
    int i;
    float f;
    scanf("%d %f", &i, &f); // 注意这里的空格
    printf("i = %d, f = %f\n", i, f);
    return 0;
}
```

### 输入分析：1234abc3.14\n

1. **scanf 开始工作**：它从输入流中读取字符。

2. **匹配 %d**：scanf 读取 1，是数字，继续；读取 2，是数字，继续；读取 3，是数字，继续；读取 4，是数字，继续。

到这里，它已经成功读取了 1234，这是一个有效的整数。scanf 将 1234 转换为整数，并存入变量 i 中。

状态：i 的值现在是 1234。scanf 继续读取下一个字符。

3. **处理空格**：scanf 读取下一个字符，是 a。但是，格式字符串中 %d 和 %f 之间有一个空格。scanf 的规则是：它会跳过（忽略）所有空白字符（空格、换行、制表符等），直到找到第一个非空白字符。

所以，scanf 看到输入流中的 a，它知道 a 不是空白字符，于是它停止跳过，准备开始匹配 %f。

4. **匹配 %f**：scanf 试图将 a 解释为一个浮点数。但 a 不是一个有效的数字字符（+，-，. 或 0-9）。读取失败：scanf 无法将 a 转换为浮点数。它立即停止处理，不再读取后续的 bc3.14。

### 最终结果：

scanf 成功读取并赋值了 1 个变量（也就是 i）（在处理 %f 时失败了）

i 的值是 1234；f 的值是未定义的（或者说是随机的，f 的内存没有被正确赋值）

②想在输入 1234abc3.14\n 的时候，输出变为 1234 和 3.14，可采取 **fgets + sscanf** 先读取整个字符串，再进行解析，方法：

1. **用 fgets 读取一整行**：将用户输入的整个字符串（包括 1234abc3.14）读入一个字符数组中。

2. **用 sscanf 从字符串中解析**：使用 sscanf 从读入的字符串中，按照格式 ("%d %f") 提取整数和浮点数。

此方法的好处是，sscanf 的工作对象是内存中的一个字符串，而不是实时输入流，所以行为更加可控。

修改后的程序代码：

```
#include <stdio.h>
#include <string.h> // 用于 strlen 函数

int main() {
    int i;
    float f;
    char input[100]; // 定义一个足够大的字符数组来存储输入

    printf("请输入数据：");
    // 1. 使用 fgets 读取一整行输入
    fgets(input, sizeof(input), stdin);

    // 2. 使用 sscanf 从 input 字符串中解析数据，sscanf 的用法和 scanf 几乎一样，只是第一个参数是字符串
    int ret = sscanf(input, "%d %f", &i, &f);

    if (ret == 2) { // 3. 检查解析是否成功
        printf("解析成功！\n");
        printf("i = %d, f = %f\n", i, f);
    } else {
        printf("解析失败！未能从输入中找到有效的整数和浮点数。\\n");
        printf("你输入的是：%s\\n", input);
    }

    return 0;
}
```

运行分析：输入 1234abc3.14\n

1. **fgets** 的工作：读取输入的整个字符串 1234abc3.14\n，并将其存入 **input** 数组中。

input 现在的内容是："1234abc3.14\n"（包含换行符）。

2. **sscanf** 的工作：**sscanf** 开始从 input 字符串中读取。它匹配 %d，成功读取 1234，并存入 i。它匹配空格，并跳过。它匹配 %f，成功读取 3.14，并存入 f。它成功地从字符串中提取出了两个值。

3. **printf** 的输出：

ret 的值是 2（因为成功解析了 2 个变量）。

程序会输出：

1        解析成功！

2        i = 1234, f = 3.140000

为什么这能成功？

fgets 把所有输入都“捕获”了，不管它是什么。

sscanf 在处理时，会从字符串的开头开始，按照格式要求去匹配。当它匹配到 3.14 时，它已经跳过了前面的 1234 和 abc，因为 abc 不符合 %f 的要求，所以 sscanf 会忽略它（**sscanf** 并不是“跳过” a、b、c，而是“消耗”掉它们，然后继续尝试匹配 %f），继续寻找下一个符合浮点数格式的部分，并成功找到了 3.14。

这种方法是 C 语言中处理这种“脏数据”输入的标准做法。

## 第一部分：fgets 的用法和参数

fgets 函数的作用是从指定的输入流（通常是键盘 stdin）中读取一行文本，并将其存储到字符数组中。它的函数原型是：

```
char *fgets(char *str, int n, FILE *stream);
```

让我们来解析这三个参数：

### 1. char \*str（第一个参数）

含义：这是一个指向字符数组的指针，也就是你用来存储输入字符串的“容器”。

在你的代码中：input 是一个字符数组，&input[0] 是它的首地址。所以 fgets(input, ...) 就是告诉函数：“请把读到的内容存到 input 这个数组里。”

### 2. int n（第二个参数）

含义：这是要读取的最大字符数（包括结尾的换行符 \n）。

在你的代码中：sizeof(input) 会计算出 input 数组的大小（例如，如果 input 定义为 char input[100];，那么 sizeof(input) 就是 100）。这确保了 fgets 不会读取超过数组容量的字符，从而防止缓冲区溢出，这是一个非常重要的安全措施。

### 3. FILE \*stream（第三个参数）

含义：这是输入流的指针，指定从哪里读取数据。

在你的代码中：stdin 是一个标准输入流的指针，它代表键盘。所以 fgets(..., stdin) 的意思就是“从键盘读取数据”。

总结一下 fgets(input, sizeof(input), stdin); 的意思：

“从键盘读取最多 sizeof(input) 个字符，并将它们存入 input 数组中。”

## 第二部分：%d 和 %f 在 scanf 与 sscanf 中的行为差异

有一个关键点需要澄清：**sscanf** 并不是“跳过” a、b、c，而是“消耗”掉它们，然后继续尝试匹配 %f。

让我们用例子 1234abc3.14 来详细分析：

情况一：使用 **scanf**（从实时输入流读取）

代码：scanf("%d %f", &i, &f);

scanf 开始读取。

它匹配 %d，成功读取 1234，i 被赋值为 1234。

它匹配空格，并跳过。

它开始匹配 %f。它读取下一个字符 a。

a 不是一个有效的浮点数字符（+，-，.，0-9）。

scanf 立即停止，并返回 1（表示只成功读取了 1 个变量）。它不会再看后面的 bc3.14。

结论：scanf 在遇到不符合预期的字符时，会中断整个读取过程。

## 情况二：使用 sscanf（从内存字符串读取）

```
sscanf(input, "%d %f", &i, &f);
```

（假设 input 的内容是 "1234abc3.14\n"）

sscanf 开始从 input 字符串的开头读取。

它匹配 %d，成功读取 1234，i 被赋值为 1234。

它匹配空格，并跳过。

它开始匹配 %f。它读取下一个字符 a。

a 不是一个有效的浮点数字符。但是，sscanf 的行为是：它不会中断，而是会尝试继续在字符串中寻找下一个符合 %f 格式的部分。

sscanf 会“消耗”掉 a，然后看 b，b 也不符合，再消耗掉 b，看 c，c 也不符合，再消耗掉 c。

当它消耗掉 c 后，下一个字符是 3。3 是一个有效的数字，sscanf 成功地匹配了 %f，并读取了 3.14，f 被赋值为 3.14。

sscanf 成功地匹配并赋值了 2 个变量，所以它返回 2。

**结论：**sscanf 在处理内存中的字符串时，会尝试从当前位置继续向后寻找，直到找到符合格式说明符的部分，或者到达字符串末尾。它不会因为中途遇到不匹配的字符而立即停止整个解析过程。

所以，你的理解基本正确：sscanf 会“跳过” a、b、c，然后发现 3.14 符合 %f 的格式，从而成功提取出 3.14。这正是它能处理 1234abc3.14 这种“脏数据”的原因。

对应 p11 “分区 新分区 1 的第 42 页”

## 为什么这很有用？

%c 默认会读取输入流中的下一个字符，包括空格和换行符。这有时会导致不符合预期的结果。

例子：

```
char c1, c2;
scanf("%c", &c1); // 假设你输入 "a b"
scanf("%c", &c2);
c1 会得到 'a'。
```

scanf 在读取 a 后，下一个字符是空格。所以 c2 会得到 ' '（空格）。

而使用空格的技巧：

```
char c1, c2;
scanf("%c", &c1); // 假设你输入 "a b"
scanf(" %c", &c2); // 注意 %c 前面的空格
c1 会得到 'a'。
```

scanf 在读取 a 后，会处理格式字符串中的空格，跳过输入流中的空格。

然后，%c 会读取下一个非空白字符，也就是 'b'。所以 c2 会得到 'b'。

## 总结：

“只有%控制符”：是 scanf 最常见的用法，格式字符串中只包含 %d，%f，%c 等说明符。

“在%c 前面加空格”：是一种特殊的技巧，用于让 %c 跳过前导的空白字符，只读取第一个有意义的字符，这在处理用户输入时非常有用。